

```
# 4. Execute a command and store the output to a given file
execute 'my_command' do
  command 'ls -lR /home/raju/lab_demo | grep ^d | wc -l > /home/raju/totdir'
end
```

Figure 3.4: Code for command execution

```
# 5. To execute shell script
script 'execute_given_script' do
  interpreter 'bash'
  user 'root'
  cwd '/home/raju'
  code <<-EOH
  ./myscript.sh
  # Add more scripts, if you want to be executed...
EOH
end
```

Figure 3.5: Code for shell script execution

```
# 6. To copy a file content
file '/home/raju/testtcp.txt' do
  content IO.read('/home/raju/test.txt')
  action :create
end
```

Figure 3.6: Code for copying a file

```
# 7. To create a backup
bash 'take_backup' do
  user 'root'
  cwd '/home/raju'
  code <<-EOH
  tar cvf temp.tar *
  gzip temp.tar
  mv temp.tar.gz /backup
EOH
end
```

Figure 3.7: Code for taking a backup

If you want to execute a command to find the total number of directories in a given path, the code example in Figure 3.4 can be used. 'Execute' is the resource type, and the command line specifies the command, which is to be executed. The command `ls -lR` lists all the files, including sub-directories in the `lab_demo`. `grep ^d` filters only directories and `wc -l` will give the total number of directories, while the output will be stored in the `totdir` file.

In a given scenario, we may need to execute certain existing shell scripts in all the nodes. The code example given in Figure 3.5 will help us to do it. Script is the resource type, `execute_given_script` is the resource name, and we need to specify which shell interpreter should be used to execute the script since the syntax of a shell script is different in between shells. The available shells in a system are listed in the `/etc/shells` file. Apart from shell script, you can also execute Perl, Python or Ruby scripts. In our

```
# 8. To stop iprdump service
service 'iprdump' do
  action :stop
end
```

Figure 3.8: Code for stopping a service

```
# 7. To create a backup
bash 'take_backup' do
  user 'root'
  cwd '/home/raju'
  code <<-EOH
  tar cvf temp.tar *
  gzip temp.tar
  mv temp.tar.gz /backup
EOH
end
```

Figure 3.9: Code for setting up a static IP address

```
# 10. Create user account
user 'newuser' do
  home '/home/newuser'
  shell '/bin/bash'
end
```

Figure 3.10: Code for user creation

```
# Execute chef-client command in every 30 minutes
cron 'chef-client' do
  minute '30'
  user 'root'
  command 'chef-client >> /home/raju/chef_client_log'
  action :create
end
```

Figure 3.11: Code for cron job creation

example, we use the Bash shell as the interpreter, the user is the root, current working directory (cwd) is `/home/raju` and the code `<<-EOH` is the starting block of a source code file, which ends with `EOH`. We can specify the scripts to be executed inside the block of source code.

The code example given in Figure 3.6 explains how to copy the content of a file `test.txt` into a new file `testcp.txt`. The resource type is `file`, and `IO.read` will read the contents from `test.txt`. The action is to create the `testcp` file.

Taking backups of important files is an important task for a systems administrator. The code given in Figure 3.7 shows how to do it through a cookbook. Bash is the resource to do certain tasks like installing packages or executing blocks of commands. Here, `take_backup` is the name of the resource block. We group all the files into `temp.tar` using the `tar` command in the `/home/raju` directory, and then compress it with the `gzip` command. The created backup file `temp.tar.gz` is moved into `/backup`.

To set up a Web server or ftp server, we first need to install the necessary packages and start the service. The code